

Automated Rating Review System

1. Project Overview

An automated review system analyzes customer reviews and predicts the corresponding star ratings. Natural language processing (NLP) is used to extract sentiment and key features from the review texts.

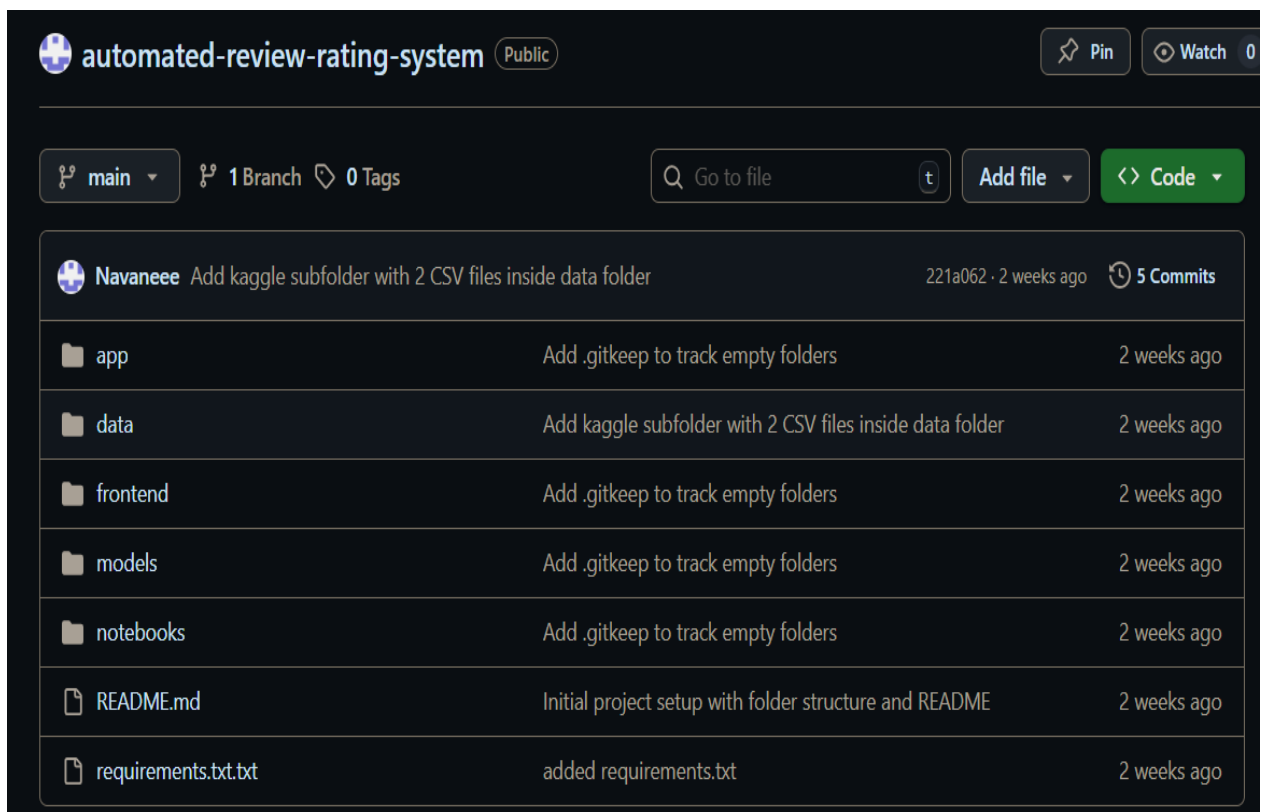
2. Environment Setup

- Installed Python 3.13.3 and the required libraries such as pandas, numpy, matplotlib, seaborn, and scikit-learn. For data preprocessing, NLP, and visualization, these packages were utilized.
- VS Code is used as the IDE.

3. GitHub Project Setup

Created a GitHub repository named automated-review-rating-system.

Structure of directory.



The screenshot shows the GitHub interface for the repository 'automated-review-rating-system'. The repository is public and has 1 branch and 0 tags. The commit history shows a commit by Navaneee 221a062, 2 weeks ago, with 5 commits in total. The file list includes folders 'app', 'data', 'frontend', 'models', and 'notebooks', each with a commit message 'Add .gitkeep to track empty folders' and a timestamp of '2 weeks ago'. It also includes files 'README.md' (Initial project setup with folder structure and README, 2 weeks ago) and 'requirements.txt.txt' (added requirements.txt, 2 weeks ago).

File/Folder	Commit Message	Commit Hash	Time
app	Add .gitkeep to track empty folders	221a062	2 weeks ago
data	Add kaggle subfolder with 2 CSV files inside data folder	221a062	2 weeks ago
frontend	Add .gitkeep to track empty folders	221a062	2 weeks ago
models	Add .gitkeep to track empty folders	221a062	2 weeks ago
notebooks	Add .gitkeep to track empty folders	221a062	2 weeks ago
README.md	Initial project setup with folder structure and README	221a062	2 weeks ago
requirements.txt.txt	added requirements.txt	221a062	2 weeks ago

4. Data Collection

The dataset was collected from Kaggle and it consists of 568454 Amazon product ids, customer reviews, star ratings and 7 more features also.

Dataset link : https://github.com/Navaneeee/automated-review-rating-system/tree/main/data/kaggle/review_data.csv

5. Data Preprocessing.

5.1 Data Preview and Dropping Unnecessary Columns

The Amazon review data consisted of 10 different columns. It preprocessed to include only review ratings and review text. All other columns were removed. The preview is given below. (568454, 2) is the dimension of the new data.

```
data.head()
```

	rating	review_text
0	5	I have bought several of the Vitality canned d...
1	1	Product arrived labeled as Jumbo Salted Peanut...
2	4	This is a confection that has been around a fe...
3	2	If you are looking for the secret ingredient i...
4	5	Great taffy at a great price. There was a wid...

5.2 Missing Values

There were no missing values present in the data.

5.3 Duplicate Values Handling

There are 174779 number of duplicate rows were present in the data and all of them were removed with the code

```
data.drop_duplicates(inplace=True)
Also 96 duplicate reviews were present in review text column.Those were also
removed with the code.
data.drop_duplicates(subset=['review_text'], inplace=True)
```

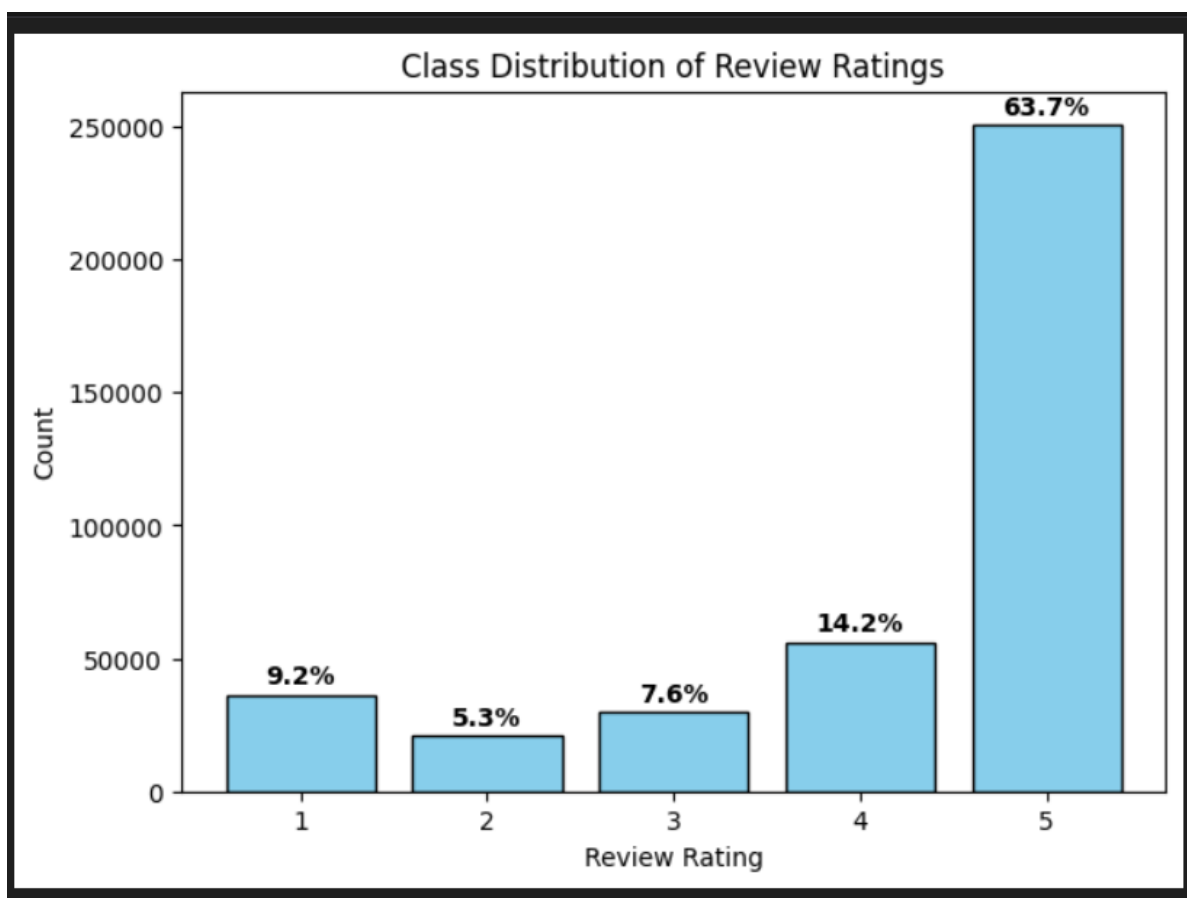
The final shape of the data was (393579, 2)

5.4 Class Distribution

The total value count of each rating is found with using `value_count()` method and it shown as

```
Class Distribution (Counts):  
rating  
1      36275  
2      20792  
3      29754  
4      56042  
5     250716  
Name: count, dtype: int64
```

The distribution of ratings is then plotted with a barplot.



The inferences from the barplot were;

1. The dataset is heavily imbalanced.
2. 5-star reviews dominate and it's around 64%
3. 2-star reviews are very low, around 5%
4. To make the data balance, we try random downsampling to the high ratings, 250,716 ratings are reduced to 50,000 and all other ratings are increased to 50,000.

5.5 Splitting the dataset to Balanced data and Imbalanced data.

The whole data is split into a balanced data with the code

```
#define the sample size(half of class 2 star ratings' count)
sample_size=class_counts[2]//2

#Sample from rating 2
#Selecting all the rows where rating column =1 and sample size is half of lowest
count class
df2_list=data[data['rating']==2].sample(n=sample_size,random_state=42) #ensures
the sampling is reproducible

#Sample from rating 1,3,4,5
#selecting same number of sample size rows from all other ratings' class
dfr_list=[data[data['rating']==r].sample(n=sample_size,random_state=42) for r in
[1,3,4,5]]

#concatenate df2_list with all others( df2,[df1,df3,df4,df5])
balanced_dataset = pd.concat([df2_list] + dfr_list)

#Create second dataset consisting of remaining rows
second_dataset = data.drop(balanced_dataset.index)

#Shuffle the combined dataset
balanced_dataset = balanced_dataset.sample(frac=1,
random_state=42).reset_index(drop=True)

#Saving the balanced dataset.
balanced_dataset.to_csv(r"C:\Users\kjinav\Documents\Navaneet\automated-review-
rating-system\data\cleaned\balanced_data.csv", index=False)
```

And split into an imbalanced dataset with code;

```
#setting the target percentages
percentage_dict={1:0.10,      # 1 *      =10%
                2:0.15,      # 2 **     =15%
                3:0.25,      # 3 ***   =25%
                4:0.30,      # 4 ****  =30%
                5:0.20}      # 5 ***** =20%

#total number of rows in the dataset
total=len(second_dataset)

#create an empty list to keep new data
new=[]

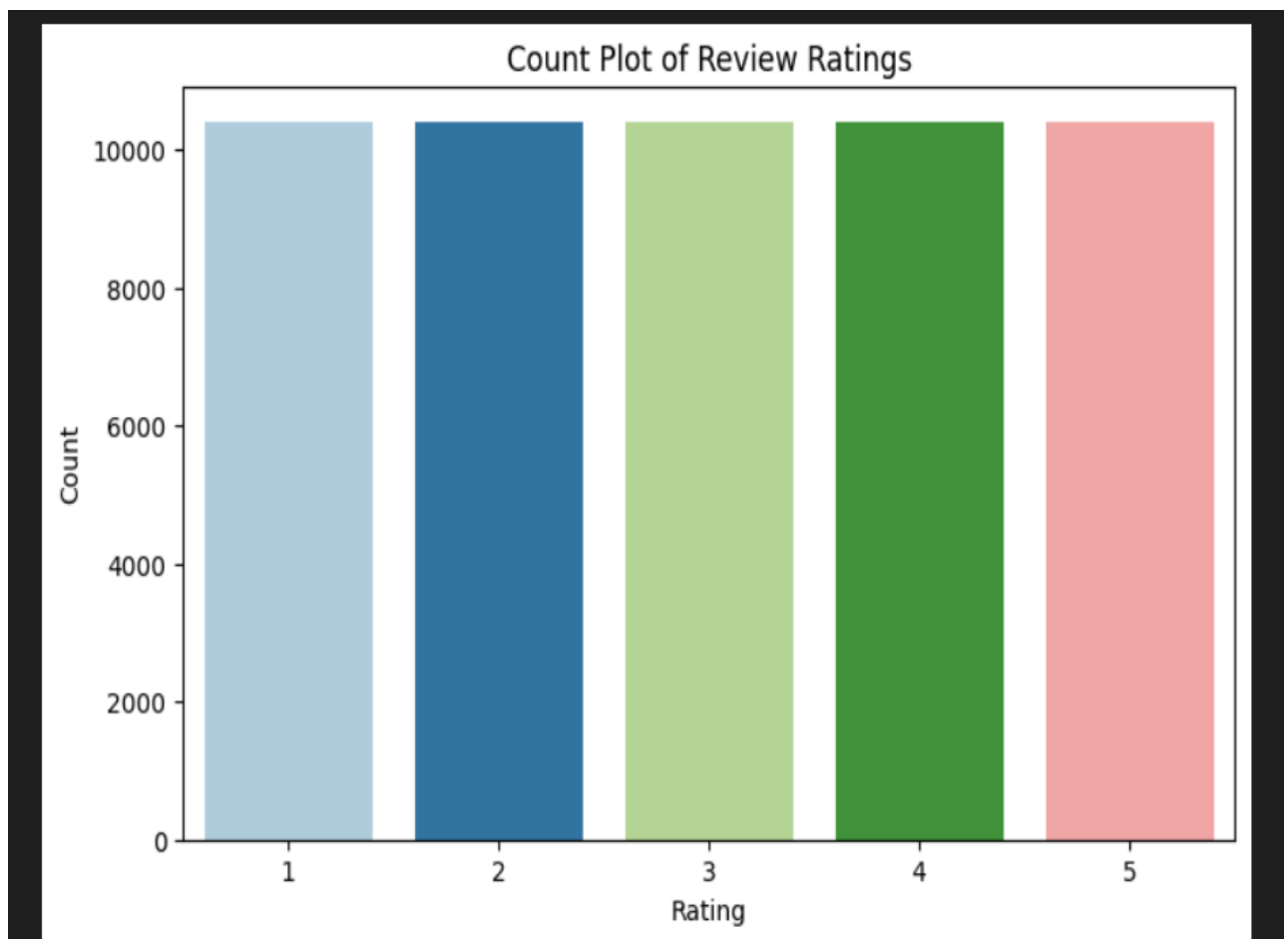
#for each ratings,we take number of rows we want according to the target
percentage
for ratings,percentage in percentage_dict.items():
    total_rows_needed=int(round(percent*total))
```

```
rows=second_dataset[second_dataset['rating']==ratings]
#select only that rating(filtering the rows)
sample_rows=rows.sample(total_rows_needed,replace=True,random_state=42)
new.append(sample_rows)
```

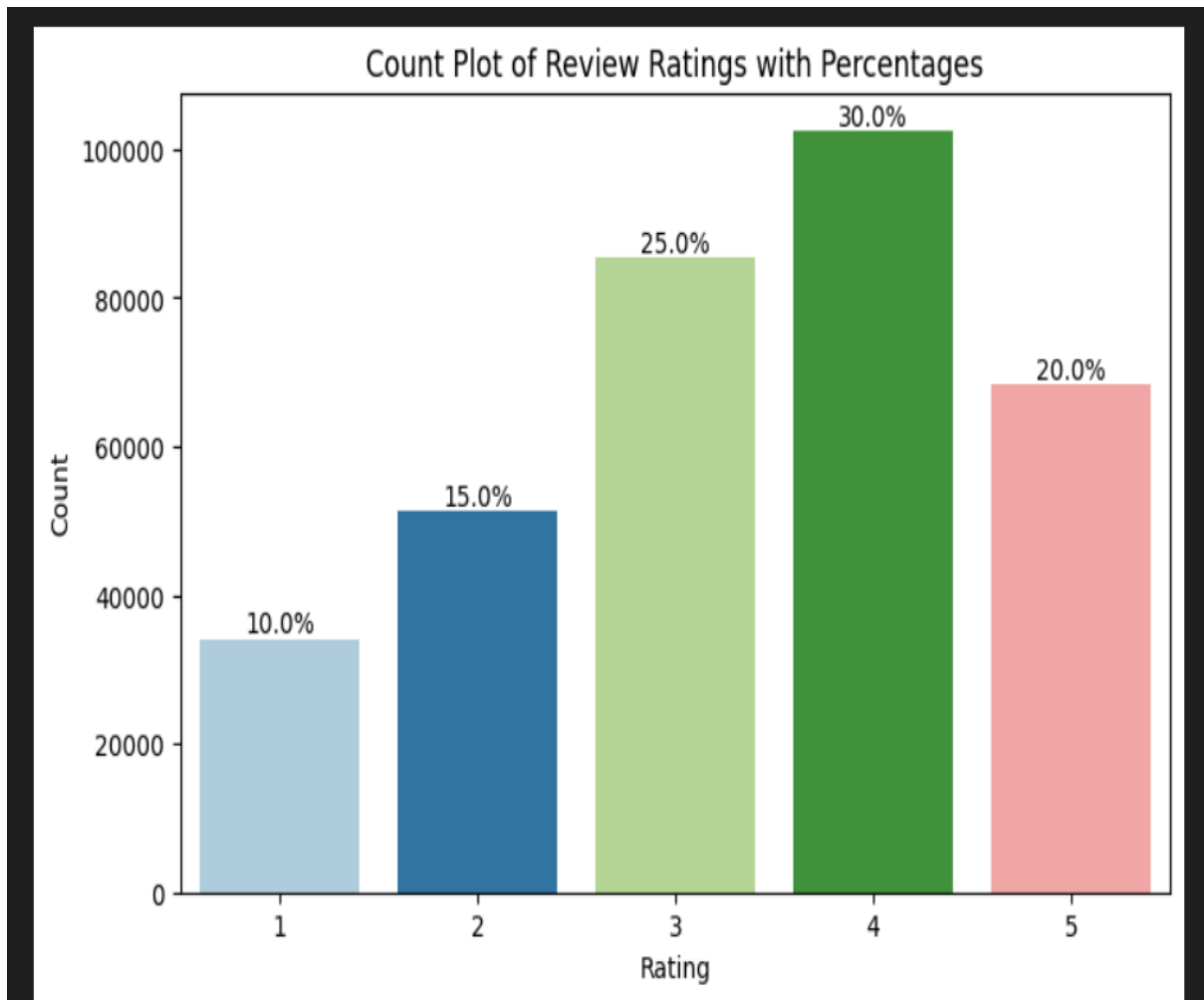
Thereafter saved,naming imbalanced_data

```
#save this imbalanced dataset.
imbalanced_data.to_csv(r"C:\Users\kjinav\Documents\Navaneee\automated-review
rating-system\data\cleaned\imbalanced_data.csv", index=False)
```

The countplot of balanced data;



The countplot of imbalanced data



6. Balanced Dataset

Dataset link : https://github.com/Navaneee/automated-review-rating-system/tree/main/data/kaggle/balanced_data.csv

6.1 Data Import and Preprocessing

This stage involves importing the dataset, inspecting its structure, handling missing values, and performing initial cleaning steps such as removing duplicates and computing word counts.

Data Loading

```
data=pd.read_csv(r"C:\Users\kjav\Documents\Navaneee\automated-review-rating-system\data\cleaned\balanced_data.csv")
```

The preview of the data just given.

	rating	review_text
0	5	Love this product, however the Amazon price is...
1	4	This has recently become my staple breakfast c...
2	4	This spread is just right but costs only about...
3	2	I guess I learned from this purchase that rais...
4	2	This coffee is sort of grocery store quality-a...

Importing all the necessary libraries

The following Python libraries were used for text preprocessing, feature extraction, and dataset splitting.

Library	Purpose
string	Contains string constants and utility functions. Used to remove punctuation from text.
nltk	Natural Language Toolkit for NLP tasks such as tokenization, stemming, and lemmatization.
from nltk.corpus import stopwords	Provides a list of common stopwords (e.g., "the", "is", "and"), which are removed to reduce noise.

Library	Purpose
from nltk.stem import WordNetLemmatizer	Reduces words to their base or dictionary form (lemmatization).
from sklearn.model_selection import train_test_split	Splits datasets into training and testing sets for model evaluation.
from sklearn.feature_extraction.text import TfidfVectorizer	Converts textual data into numerical features using TF-IDF, representing the importance of words across documents.

6.2 Text Preprocessing

Created a user defined function called “preprocess_text”. This function cleans and prepares the review text for analysis using Natural Language Processing (NLP) techniques.

1. Lowercasing:

Converts all text to lowercase to maintain consistency.

2. Removing URLs and HTML tags:

Eliminates links and HTML elements that are not useful for text analysis.

3. Removing emojis and special characters:

Keeps only alphabetic characters to focus on meaningful words.

4. Tokenization:

Splits the text into individual words (tokens).

5. Stopword Removal:

Commonly used words that do not contribute much to the meaning or sentiment of a sentence are removed. The list of stopwords used comes from **NLTK's** default English stopwords set. Removing these words helps reduce noise and improves the focus on meaningful terms for tasks like text classification, sentiment analysis, or information retrieval. **NLTK** provides a simple and lightweight stopwords list ready to use, making it ideal for quick preprocessing in research or small projects.

spaCy also offers stopwords lists, but it is more resource-intensive and geared toward full-featured NLP pipelines. For tasks where only stopwords removal is required, NLTK is faster and easier to implement. The list of stopwords used comes from NLTK's default English stopwords set, which includes words such as:

```
{'by', 'yourself', 've', 'been', 'other', 'in', "didn't", 'each', 'this', "he'll", 'am', "they've", "you're", 'ourselves', 'ours', 'doing', "it'll", "shan't", 'hadn', 'most', 'than', 'they', "we'd", 'she', 'haven', "couldn't", "hasn't", 'his', 'out', 'over', "we'll", "isn't", 'against', 'which', 'hers',
```


'or', 'won't', 'you'd', 'above', 'should', 'on', 'because', 'he', 'she'll', 'does', 'couldn't', 'shan', 'don't', 'they'd', 'with', 'a', 'why', 'weren't', 'shouldn't', 'yourselves', 'ain't', 'she'd', 'more', 'll', 'myself', 'theirs', 'but', 'about', 'them', 'did', 'to', 'has', 'be', 'so', 'of', 'too', 'while', 'should've', 'those', 'don't', 'she's', 'some', 'all', 'such', 'aren't', 'from', 'here', 'whom', 'mightn't', 'wouldn't', 'm', 'you've', 'at', 'themselves', 'mustn't', 'if', 'then', 'now', 'nor', 'that', 'they're', 'won', 'no', 'have', 'me', 'were', 'own', 'we've', 'any', 'o', 'do', 'mightn't', 'we're', 'he's', 'shouldn't', 'further', 'was', 'there', 'is', 'its', 'between', 'for', 'only', 'an', 'my', 'i', 'haven't', 'where', 'hadn't', 't', 'what', 'into', 'once', 'below', 'himself', 'having', 'are', 'ma', 'off', 'we', 'who', 'before', 'yours', 'hasn't', 'not', 's', 'as', 'doesn't', 'didn't', 'him', 'that'll', 're', 'it'd', 'isn't', 'it', 'can', 'her', 'd', 'weren't', 'had', 'itself', 'their', 'herself', 'you'll', 'again', 'you', 'down', 'your', 'i'd', 'it's', 'through', 'under', 'very', 'wasn't', 'during', 'how', 'wasn't', 'mustn't', 'i've', 'both', 'until', 'y', 'i'm', 'and', 'the', 'after', 'when', 'few', 'aren't', 'our', 'they'll', 'up', 'being', 'i'll', 'needn't', 'these', 'same', 'will', 'wouldn't', 'he'd', 'just', 'doesn't', 'needn't'}

6. Lemmatization

Lemmatization is the process of converting each word to its base or dictionary form (known as the lemma). For example:

“walking” → “walk”,

“better” → “good”.

This step helps ensure that words with similar meanings are treated as the same during analysis, thereby improving model consistency and reducing feature sparsity. While both **stemming** and lemmatization aim to reduce words to their root form, lemmatization is more accurate and linguistically meaningful.

7. Rejoining:

The cleaned tokens are combined back into a single string. The cleaned text is saved in a new column “cleaned_review”, and a “word_count” column is created to store the number of words in each processed review.

8. Removed too long or short texts:

The review texts above word below 3 and above 175 is removed. The codes given below.

```
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

def preprocess(text):

    # lowercase
    text = text.lower()

    # remove URLs
    text = re.sub(r"http\S+|www\S+", "", text)

    # remove HTML tags
    text = re.sub(r"<.*?>", "", text)
```

```

# remove emojis and non-alphabetic characters
text = re.sub(r"[^a-z\s]", "", text)

# tokenize
tokens = text.split()

# remove stopwords & lemmatize
tokens = [lemmatizer.lemmatize(word) for word in tokens if word not in
stop_words]

# rejoin
return " ".join(tokens)

# Applying preprocess function to review text column.
data["cleaned_review"] = data["review_text"].astype(str).apply(preprocess)

#finding count of words in review text
data["word_count"] = data["cleaned_review"].apply(lambda x: len(x.split()))

```

6.3 Data Visualization

A subplot consisting of a barplot ,histogram and box[plot of word count by ratings plotted.

```

#subplots of 3 rows and 1 column
fig, axes = plt.subplots(3, 1, figsize=(10, 15))

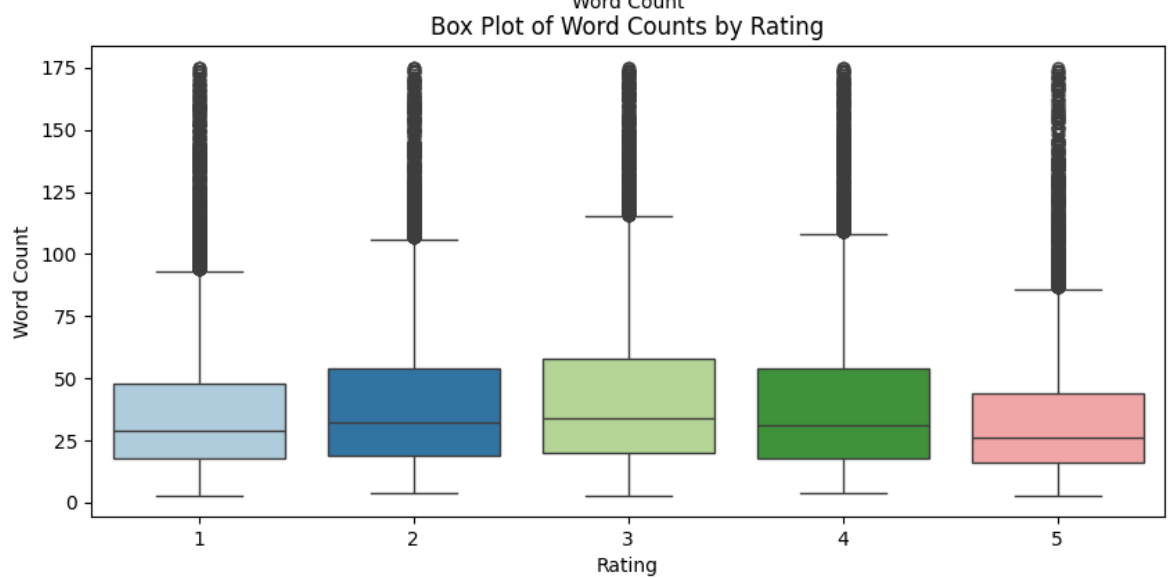
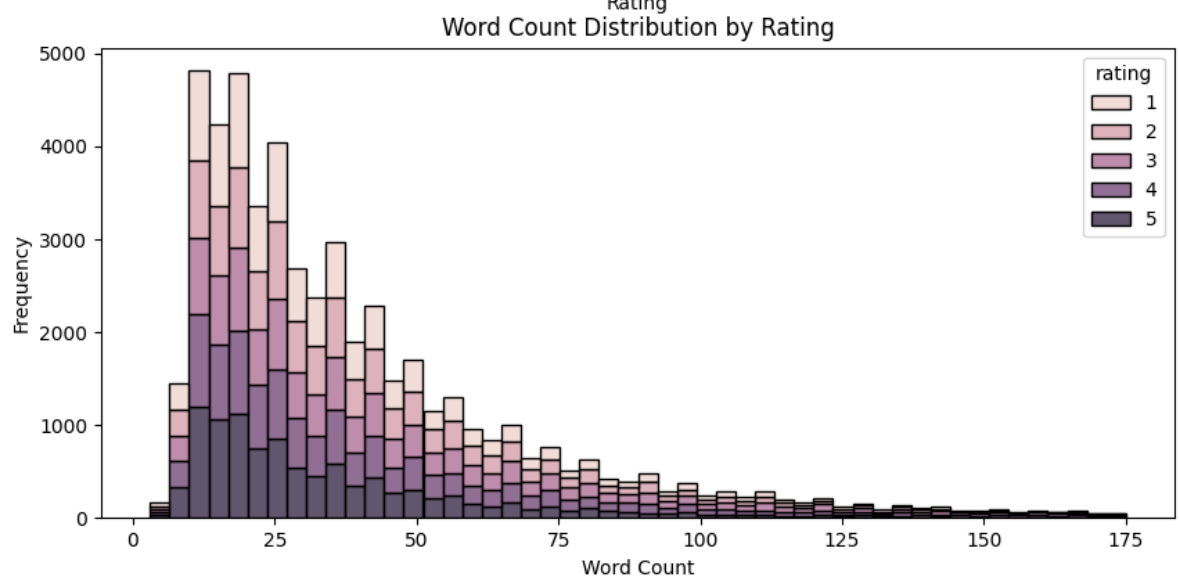
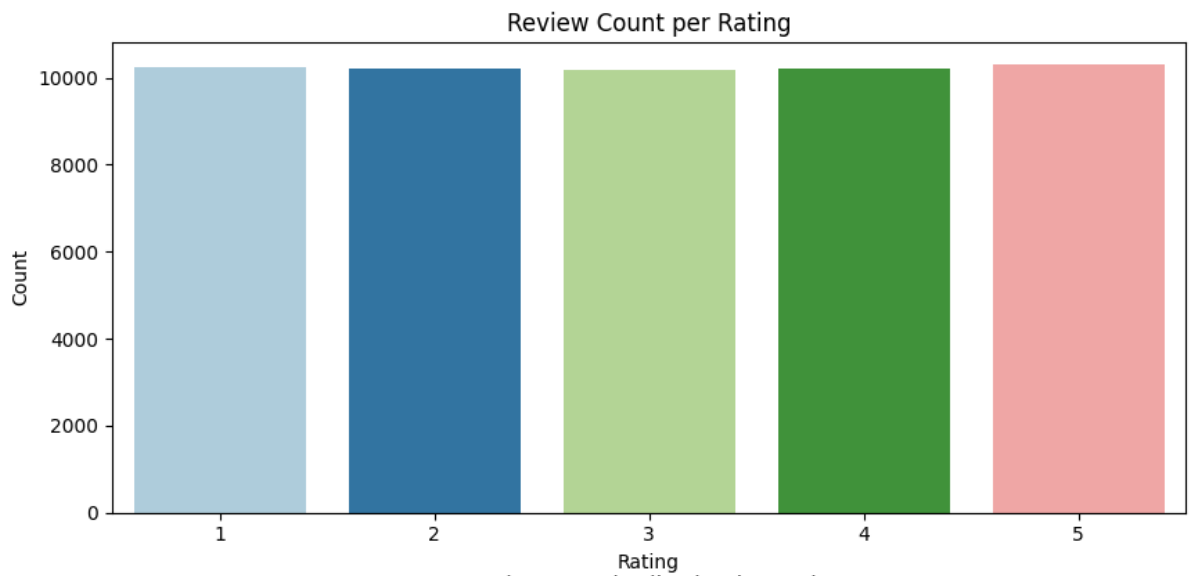
#bar plot: Review count per rating
sns.countplot(x="rating", data=data, palette="Paired", ax=axes[0])
axes[0].set_title("Review Count per Rating")
axes[0].set_xlabel("Rating")
axes[0].set_ylabel("Count")

#histogram of word counts
sns.histplot(data, x="word_count", hue="rating", multiple="stack", bins=50,
ax=axes[1])
axes[1].set_title("Word Count Distribution by Rating")
axes[1].set_xlabel("Word Count")
axes[1].set_ylabel("Frequency")

#box plot of word counts
sns.boxplot(x="rating", y="word_count", data=data, palette="Paired", ax=axes[2])
axes[2].set_title("Box Plot of Word Counts by Rating")
axes[2].set_xlabel("Rating")
axes[2].set_ylabel("Word Count")

plt.tight_layout()
plt.show()

```



The inferences from the plots are;

1. Barplot
 - a. The bar heights are same, means balanced the dataset across all ratings from 1 to 5.
2. Histogram
 - a. Most reviews have fewer words and around 50. The distribution is right-skewed, large number of short reviews, and fewer long ones.
 - b. The overlap between ratings shows review length is not a strong indicator of sentiment/rating. Both good and bad reviews can be short or long.
3. Boxplot
 - a. Median word counts are around 25–35 words
 - b. The interquartile ranges are also similar, meaning review length doesn't vary much between 1-star and 5-star ratings.
 - c. Many outliers exist (long reviews above 100 words). These could be detailed opinions or complaints.
 - d. Slightly shorter reviews appear in extreme ratings (1 and 5), while middle ratings (2 to 4) show more spread, possibly reflecting more nuanced feedback.
 - e. Outliers which are long reviews may carry rich detail, potentially useful for deeper analysis of sentiment.

6.4 Sample ratings preview

5 Sample reviews of 5 ratings are printed with the code

```
# preview of sample reviews per rating
for r in sorted(data["rating"].unique()):
    print(f"sample review of rating {r}")
    display(data[data["rating"] == r]["cleaned_review"].sample(min(5,
len(data[data["rating"] == r]))).to_list())
```

sample review of rating 1

1. 'wondering much cheaper flavor know cat wouldnt touch acted like stunk backed away he went eat day old dried cat food rather eat junk im concerned someone wrote might tainted recalled later well ill save case get recalled maybe get refund dont like amazon wont let return food items unless recalled especially unopened cans a lot people gave good rating a lot people didnt think one cat like odds bad shouldnt buy',

2. 'ordered box new man dried apricot understood dark color lack preservative ate anyway became ill vomiting nausea chill dizziness about hour eating occur apricot thought bad virus

soi ate slow learnersame result miserable contacted amazon gracious refunding moneythe remainder apricot trash warned',

3. 'already ordered directly cafe britt jumped chance order cafe britt product amazon thus saving shipping cost even getting break price experience dealing cafe britt great ordered lb bag whole bean costa rica dark roast coffee loved wanting try something different ordered lb bag tres rio valdivia whole bean coffee thru amazomcom upon opening first bag noticed difference quality quite apparent assumed probably different type coffee opening second

4. bag think looking freshness date packaging ordered coffee july received august rd package freshness date jun lot appears amazon shipped coffee way date know local grocery store going sell food date amazon itsecond entryhaving spoken customer service rep amazon told outofdate product replaced cost appreciate must commend amazon responsiveness complaint',

5. 'ordered product received broken bottle amazon excellent reshipped entire order one day shipping unfortunately even cent bottle overpaid cheese taste really salty mess bottle average product save money'

sample review of rating 2

1. 'parachute coconut oil really coconut kind hard tell coconut oil smell like sort bacon fat tried use dont like smell bacon product totally bad',

2. 'mom turned alba milkshake remember tasting pretty good thrilled find amazon year ordered item massive quantity im trying find people give since cant force fake aftertasteladen stuff throat dont know different ingredient faulty memory stuff terribly disappointing borderline disgusting',

3. 'dog loved taste dog food didnt agree system month old lab puppy amount food recommended much food dog think stuff expensive also',

4. 'coffee bean purchased directly roaster typically arrive hour time roasted always ship within hour roasted inexpensive single sourced precisely roasted bean brewed correctly produce unbeatable cup coffeewhen purchased amazon unfortunately bean arrived day roasted date day roasted printed bag format dayoftheyearyear bag amazon came printed meaning st day june today august shipped day ago coffee peak moment come roaster decline every minute via dissipation volatile aromatics oxidation staling buy directly roaster want fresh buy amazon want cheap mediocre halfstale coffee bulk',

5. 'listing say milk chocolate covered however received dark chocolate covered peanut really dont like dark chocolate much ended giving away part ate theyre alright listed']

sample review of rating 3

1. 'ingredient dog food look pretty good suspect thing like corn gluten meal animal digest ew food coloring cant recommend liked made poop often runny going back proplan',

2. 'product arrived time correct flavor type problem though opened brown shipping box looked like whoever shipped opened product took one pouch pouch pack kind would set grocery store also ordered different flavor time company box also missing pouch total screwed pouch gum didnt bother sending product back complaining let face trouble wasnt worth gum worth still kind disappointing someone would steal customer',
3. 'area many option instant oatmeal gluten free one one option ok however one need add cinnamon',
4. 'bought friend dont know able make yet really careful eat only gluten free food',
5. 'meal tasty filling quite healthy berry really compliment crunchy spice rice however easy take on the go might think water need boiling added pilaf need another container heat water always easy find youre taking meal go found work well use hot water water cooler coffee pot'

sample review of rating 4

1. 'im bit licorice fanatic obviously else buy case say good mass produced licorice better specialty shop price everyday snacking good licorice just dont know licorice great tasking candy also good settling nerve sometimes itll make upset stomach little less upset maybe psychosomatic seem work',
2. 'cat loved liver treat packaging tore shipping patch packet',
3. 'taste good difficult dissolve even cold water need stirred whisked quite powder dissolve best dissolved cold water clump even water lukewarm',
4. 'family truly delight eating jyoti saag delicious satisfying part meal id feel better however feeding jyoti saag family reason believe can dont leach bpa saag',
5. 'great mix used many baking item pancake muffin turn good simple make also water added rather milk especially easy use']

sample review of rating 5

1. 'officially new shampoo conditioner combo smell absolutely incredible so fresh clean husband always compliment nice hair smell soft make hair shiny manageable rotate shampoo every day keep hair drying week use antidandruff shampoo condition every day dont use lot heat styling put product hair finding shampoo conditioner let wash hair air dry leaving soft laying nicely smelling great huge hair color treated prefer way length conditioner feel hair',
2. 'purchased sister holiday loved generous portion wonderful variety especially hard time thoughtful gift send love service excellent way delivery thank',
3. 'lingonberry great rotisserie chicken almost impossible fine supermarket even whole food doesnt brand consistently great quality',

4. 'marmite aquired taste slightly salty yeasty spread use toast sandwich supposed good people eat like tastesmarmite isnt easy find u get larger health food type chain like sun harvest farm whole food market charge g jar one would consider charged shipping well product price deciding whether ordering marmite online value',

5. 'product calorie low one would expect icy grainy dessert opposite fact smooth creamy flavor taste greatwe kind thing arctic zero make treat rival ice creamextra peanut buttery chocolatemix tablespoon pb little splenda enough hot water make saucy consistancy pour chocolate chocolate peanut butter arctic zero heavenpistachiocrush cup shelled pistachio piece mixin along drop almond extract vanilla arctic zero yumtropical strawberryheat crushed pineapple little splenda pour strawberry arctic zerotropical iiheat cup apple juice tablespoon splenda teaspoon cornstarch add sliced banana disk warm sauce thickened pour strawberry vanilla chocolate chocolate peanut butter arctic zerocrispy chocolate mintslightly crush packet quaker calorie pack chocolatey mint mini delight mixin vanilla chocolate alpine zerocherry jubileechop cup maraschino cherry warm juice pour mixin vanilla strawberry arctic zerothese favorite come ton eating arctic zero plain great'

7. Train Test Split

The dataset is split into training (80%) and testing (20%) sets using the `train_test_split()` function.

The `stratify=y` parameter ensures that the proportion of each rating category remains the same in both the training and test sets.

```
x = balanced["cleaned_review"] # training
y = balanced["rating"]        # testing
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
stratify=y, random_state=42)
```

8. TF-IDF Vectorization

TF-IDF is a statistical measure that evaluates how important a word is in a document relative to a collection of documents (corpus). It combines two components:

- Term Frequency (TF): Measures how often a word appears in a document.

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

- Inverse Document Frequency (IDF): Measures how unique or rare a word is across all documents.

$$IDF(t) = \log \frac{\text{Total number of documents}}{\text{Number of documents containing term } t}$$

- TF-IDF Score:

$$TFIDF(t, d) = TF(t, d) \times IDF(t)$$

- Words that occur frequently in a document but rarely across all documents get higher TF-IDF scores, highlighting their importance for classification. Common words like "the" or "is" get low scores because they appear in almost all documents.
- This TF-IDF representation allows the model to focus on the most relevant words for predicting review ratings

```
vectorizer = TfidfVectorizer(max_features=5000)
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)
print("TF-IDF Train shape:", X_train_tfidf.shape)
print("TF-IDF Test shape:", X_test_tfidf.shape)
```

#max_features=5000 limits the number of features to the top 5000 most important words in the dataset. #fit_transform() learns the vocabulary from the training data and converts it into TF-IDF features.
#transform() converts the test data into the same feature space without learning a new vocabulary.

the textual review data is converted into numerical features using “TF-IDF (Term Frequency-Inverse Document Frequency)”. Machine learning models require numerical input, so text data must be transformed into a numerical representation.

Model Training and Evaluation.

Model Selection Summary

In this project, five supervised machine learning models were developed and evaluated for sentiment classification: Logistic Regression, Linear Support Vector Machine (SVM), Naive Bayes, KNN and Random Forest.

These models were selected because they represent a balanced mix of linear, probabilistic, and ensemble learning approaches, each offering unique strengths for text-based sentiment analysis.

1. Logistic Regression:

Logistic Regression is a strong and interpretable baseline classifier for text classification tasks. It performs well with high-dimensional sparse data generated from TF-IDF or count vectorization, and provides probabilistic outputs useful for understanding class confidence levels.

2. Linear SVM:

Support Vector Machines are known for their ability to handle high-dimensional feature spaces efficiently. A Linear SVM finds an optimal separating hyperplane between sentiment classes, making it especially suitable for text data where the number of features (words) is much greater than the number of samples.

3. Naive Bayes:

Naive Bayes classifiers, particularly MultinomialNB, are commonly used in document classification and sentiment analysis due to their simplicity, speed, and strong performance on sparse text features. It assumes independence between features, which, despite being a simplification, often works well in practice for word-based models.

4. Random Forest:

Random Forests are robust ensemble models that combine multiple decision trees to improve accuracy and reduce overfitting. They were included to test whether a non-linear model could better capture complex sentiment relationships that linear models might miss.

5. K-Nearest Neighbors (KNN):

KNN is a simple, instance-based learning algorithm that classifies a sample based on the majority label of its nearest neighbors in the feature space. It was included to evaluate how a distance-based, non-parametric method performs on text data, especially when capturing local patterns in sentiment that other models might overlook.

Why These Models Were Chosen

- These models are well-established baselines for Natural Language Processing (NLP) and sentiment classification tasks.
- They handle textual feature vectors (TF-IDF) effectively.
- They offer a mix of interpretability (Logistic, SVM, Naive Bayes) and non-linear learning (Random Forest).
- Compared to deep learning methods, these models are computationally efficient and require less data, making them ideal for establishing a solid baseline before exploring more advanced architectures like LSTMs or Transformers.

Model Training and Optimization Process

Initially, all four models **Logistic Regression**, **Linear SVM**, **Naive Bayes**, and **Random Forest**, **KNeighborsClassifier** were trained using their **default parameters**. However, the accuracy was found to be **low** (ranging between **0.30 - 0.40**), indicating that the default settings and basic feature extraction were not sufficient to capture meaningful sentiment patterns.

```
# Import necessary libraries
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier

# Define models
lr = LogisticRegression(max_iter=1000)
nb = MultinomialNB()
svm = LinearSVC()
rf = RandomForestClassifier(random_state=42)
knn = KNeighborsClassifier()

# Put models into a dictionary
models = {
    "Logistic Regression": lr,
    "Naive Bayes": nb,
    "Linear SVM": svm,
    "Random Forest": rf,
    "KNN": knn,
}

# Train and evaluate
results = {}
for name, model in models.items():
    model.fit(X_train_tfidf, y_train)
    acc = model.score(X_test_tfidf, y_test)
    results[name] = acc

# Display performance summary
```

```
print("\n Model Performance Summary:")
for k, v in sorted(results.items(), key=lambda x: x[1], reverse=True):
    print(f"{k}: {round(v, 4)}")
```

Model Performance Summary:

Logistic Regression: 0.5043

Naive Bayes: 0.4927

Linear SVM: 0.4748

Random Forest: 0.4666

KNN: 0.3059

To improve performance, an **optimized TF-IDF feature extraction** approach was implemented. The following modifications were made to the TF-IDF vectorizer:

```
vectorizer = TfidfVectorizer(
    max_features=15000, # expand vocabulary      (before 5000)
    ngram_range=(1,2), # include bigrams
    min_df=3,          # ignore very rare words
    max_df=0.9         # ignore overly common words
)
# Increasing max_features and adding bigrams can capture more contextual
information.
```

This optimization helped the models capture richer text patterns, such as combinations of words (bigrams), and reduced noise from very rare or overly common words.

Hyperparameter Tuning

After improving the feature extraction, **hyperparameter tuning** was performed individually for each model using **GridSearchCV** to find the best parameter combinations that maximize model performance. The tuning process helped achieve better accuracy and balance across different sentiment classes. Cross-validation (CV) was also included to evaluate performance and ensure the model's results are stable across different subsets of the data.

```
# Hyperparameter tuning using GridSearchCV
from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(model, param_grid=params, cv=5, scoring='accuracy', n_jobs=-1)
grid.fit(X_train_tfidf, y_train)
print("Best Params:", grid.best_params_)
```

Model Implementation and Results

1. LOGISTIC REGRESSION

```
# Hyperparameter tuning
lr_params = {
    'C': [0.1, 1, 10],
    'penalty': ['l2'],
    'solver': ['lbfgs', 'liblinear'],
    'class_weight': [None, 'balanced']
}

lr_grid = GridSearchCV(
    LogisticRegression(max_iter=1000),
    lr_params,
    scoring='accuracy',
    cv=3,
    n_jobs=-1,
    verbose=1
)
lr_grid.fit(X_train_tfidf, y_train)

# Evaluate
lr_best = lr_grid.best_estimator_
y_pred_lr = lr_best.predict(X_test_tfidf)

print("\n Logistic Regression Best Params:", lr_grid.best_params_)
print("Accuracy:", round(accuracy_score(y_test, y_pred_lr), 4))
print(classification_report(y_test, y_pred_lr, digits=4))

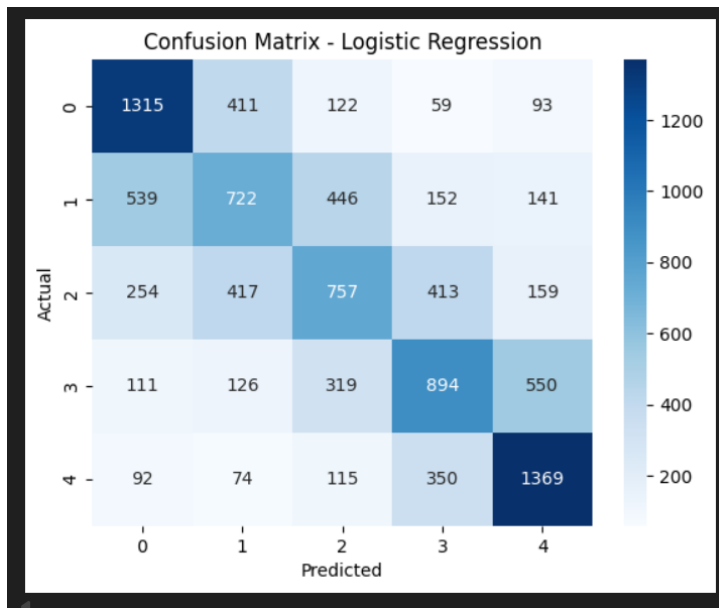
cm = confusion_matrix(y_test, y_pred_lr)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix - Logistic Regression")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

and the output was;

```
Logistic Regression Best Params:
{'C': 1, 'class_weight': 'balanced', 'penalty': 'l2', 'solver': 'liblinear'}
```

Accuracy: 0.5057

	precision	recall	f1-score	support
1	0.5690	0.6575	0.6101	2000
2	0.4126	0.3610	0.3851	2000
3	0.4304	0.3785	0.4028	2000
4	0.4786	0.4470	0.4623	2000
5	0.5921	0.6845	0.6350	2000
accuracy			0.5057	10000
macro avg	0.4965	0.5057	0.4990	10000
weighted avg	0.4965	0.5057	0.4990	10000



Logistic Regression – Inference

- After tuning, the best parameters were " C = 1, penalty = 'l2', solver = 'liblinear', and class_weight = 'balanced' "
- The model achieved an accuracy of **50.57%**. It performed better for ratings 1 and 5, showing stronger ability to detect highly negative and highly positive reviews than moderate ones.

From the confusion matrix,

- The model correctly identified a large number of 1-star (1315) and 5-star (1369) reviews.
 - Moderate ratings (2, 3, and 4 stars) are more frequently confused with nearby ratings for instance, many 3-star reviews are misclassified as 2-star or 4-star.
- This pattern indicates that the model captures the extreme sentiments effectively but struggles with mid-range or neutral reviews, where the language used is often mixed or less distinctive.

2. Linear SVM

```
# Hyperparameter tuning
svm_params = {
    'C': [0.1, 1, 10],
    'class_weight': [None, 'balanced'],
    'loss': ['hinge', 'squared_hinge']
}

svm_grid = GridSearchCV(
    LinearSVC(max_iter=2000),
    svm_params,
    scoring='accuracy',
    cv=3,
    n_jobs=-1,
    verbose=1
)
svm_grid.fit(X_train_tfidf, y_train)

# Evaluate
svm_best = svm_grid.best_estimator_
y_pred_svm = svm_best.predict(X_test_tfidf)

print("\n Linear SVM Best Params:", svm_grid.best_params_)
print("Accuracy:", round(accuracy_score(y_test, y_pred_svm), 4))
print(classification_report(y_test, y_pred_svm, digits=4))

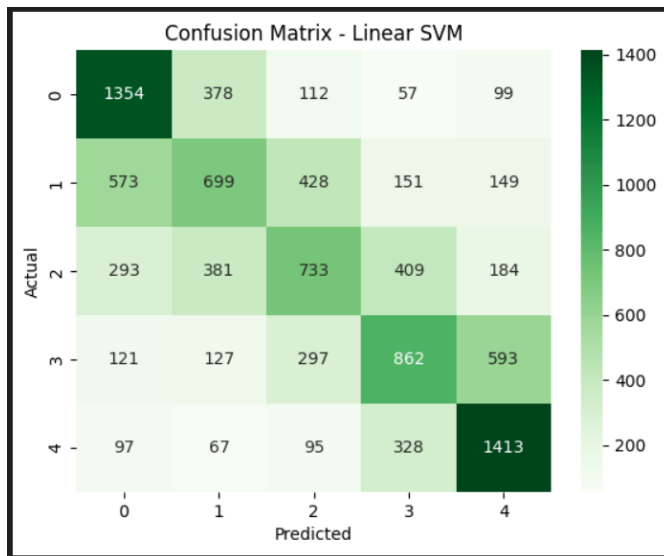
cm = confusion_matrix(y_test, y_pred_svm)
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens')
plt.title("Confusion Matrix - Linear SVM")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

and the output was;

Linear SVM Best Params:
{'C': 0.1, 'class_weight': None, 'loss': 'squared_hinge'}

Accuracy: 0.5061

	precision	recall	f1-score	support
1	0.5554	0.6770	0.6102	2000
2	0.4231	0.3495	0.3828	2000
3	0.4402	0.3665	0.4000	2000
4	0.4770	0.4310	0.4529	2000
5	0.5796	0.7065	0.6368	2000
accuracy			0.5061	10000
macro avg	0.4951	0.5061	0.4965	10000
weighted avg	0.4951	0.5061	0.4965	10000



Linear SVM – Inference

- After tuning, the best parameters were $C = 0.1$, `loss = 'squared_hinge'`, and `class_weight = None`.
- The model achieved an accuracy of **50.61%**. It performed better for ratings 1 and 5, showing stronger capability in detecting highly negative and highly positive reviews compared to moderate ones.

From the confusion matrix:

- The model correctly identified a large number of 1-star (1354) and 5-star (1413) reviews.
- Moderate ratings (2, 3, and 4 stars) are frequently misclassified as adjacent ratings. For example, many 3-star reviews are predicted as 2-star or 4-star, and 2-star reviews are often confused with 1-star or 3-star categories.

This pattern indicates that the Linear SVM captures extreme sentiments effectively, but struggles to distinguish between mid-level ratings, where the language tends to overlap or be more neutral in tone.

1. NAIVE BAYES

```
nb_params = {
    'alpha': [0.1, 0.5, 1.0, 2.0],      # smoothing parameter
    'fit_prior': [True, False]
}

nb_grid = GridSearchCV(
    MultinomialNB(),
    nb_params,
    scoring='accuracy',
    cv=3,
    n_jobs=-1,
    verbose=1
)
nb_grid.fit(X_train_tfidf, y_train)

# Evaluate
nb_best = nb_grid.best_estimator_
y_pred_nb = nb_best.predict(X_test_tfidf)

print("\n Naive Bayes Best Params:", nb_grid.best_params_)
print("Accuracy:", round(accuracy_score(y_test, y_pred_nb), 4))
print(classification_report(y_test, y_pred_nb, digits=4))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_nb)
sns.heatmap(cm, annot=True, fmt='d', cmap='Purples')
plt.title("Confusion Matrix - Naive Bayes")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

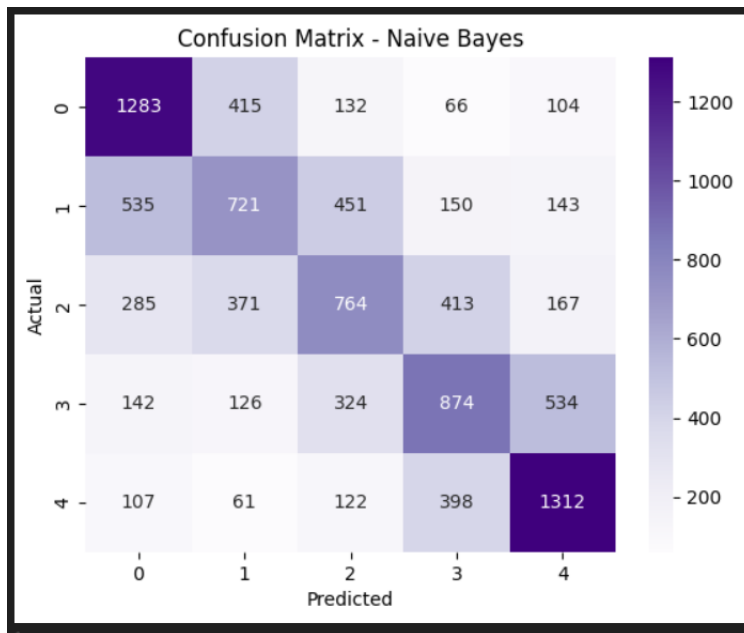
and the output was;

Naive Bayes Best Params:

{'alpha': 2.0, 'fit_prior': True}

Accuracy: 0.4954

	precision	recall	f1-score	support
1	0.5455	0.6415	0.5896	2000
2	0.4256	0.3605	0.3904	2000
3	0.4261	0.3820	0.4028	2000
4	0.4598	0.4370	0.4481	2000
5	0.5805	0.6560	0.6160	2000
accuracy			0.4954	10000
macro avg	0.4875	0.4954	0.4894	10000
weighted avg	0.4875	0.4954	0.4894	10000



Naive Bayes - Inferences

- After tuning, the best parameters were $\alpha = 2.0$ and `fit_prior = True`.
- The model achieved an accuracy of **49.54%**, which is slightly lower than the Linear SVM model but still indicates moderate performance. It performs better for extreme ratings (1 and 5) than for middle ratings (2, 3, and 4).

From the confusion matrix:

- The model correctly identified a large number of 1-star (1283) and 5-star (1312) reviews.
- Moderate ratings (2, 3, and 4 stars) are frequently confused with neighboring ratings. For example, many 2-star reviews are predicted as 1-star or 3-star, and 3-star reviews are sometimes misclassified as 2-star or 4-star.
- This shows that the Naive Bayes classifier captures extreme sentiments (strongly positive or negative) fairly well, but has difficulty distinguishing neutral or mid-range reviews, where the wording tends to overlap between classes.

1. RANDOM FOREST

```
rf_params = {
    'n_estimators': [100, 200],
    'max_depth': [10, 30, None],
    'min_samples_split': [2, 5],
    'class_weight': [None, 'balanced_subsample']
}

rf_grid = GridSearchCV(
    RandomForestClassifier(random_state=42, n_jobs=-1),
    rf_params,
    scoring='accuracy',
    cv=3,
    n_jobs=-1,
    verbose=1
)
rf_grid.fit(X_train_tfidf, y_train)

# Evaluate
rf_best = rf_grid.best_estimator_
y_pred_rf = rf_best.predict(X_test_tfidf)

print("\n Random Forest Best Params:", rf_grid.best_params_)
print("Accuracy:", round(accuracy_score(y_test, y_pred_rf), 4))
print(classification_report(y_test, y_pred_rf, digits=4))

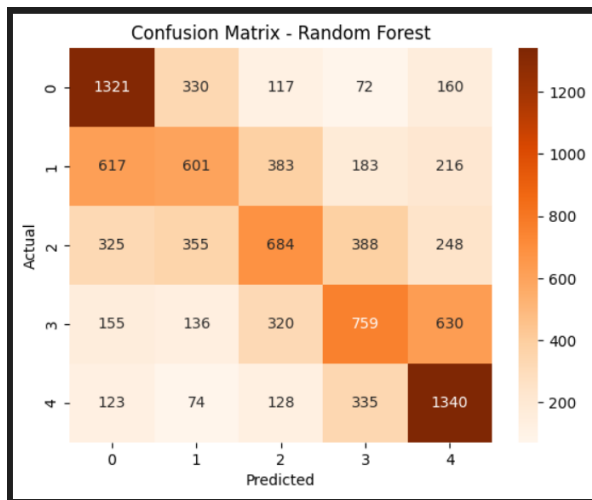
cm = confusion_matrix(y_test, y_pred_rf)
sns.heatmap(cm, annot=True, fmt='d', cmap='Oranges')
plt.title("Confusion Matrix - Random Forest")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

and the output was;

```
Random Forest Best Params:
{'class_weight': None, 'max_depth': None, 'min_samples_split': 5, 'n_estimators': 200}
```

Accuracy: 0.4705

	precision	recall	f1-score	support
1	0.5199	0.6605	0.5818	2000
2	0.4017	0.3005	0.3438	2000
3	0.4191	0.3420	0.3767	2000
4	0.4370	0.3795	0.4062	2000
5	0.5166	0.6700	0.5834	2000
accuracy			0.4705	10000
macro avg	0.4589	0.4705	0.4584	10000
weighted avg	0.4589	0.4705	0.4584	10000



Random Forest – Inferences.

- After tuning, the best parameters were `n_estimators = 200`, `min_samples_split = 5`, `max_depth = None`, and `class_weight = None`.
- The model achieved an accuracy of **47.05%**, which is slightly lower than both the Linear SVM and Naive Bayes models. It shows better performance for the extreme ratings (1 and 5) and relatively weaker results for the mid-range ratings (2, 3, and 4).

From the confusion matrix,

- The model correctly identified a large number of 1-star (1321) and 5-star (1340) reviews.
- Moderate ratings (2, 3, and 4 stars) are frequently misclassified as nearby ratings, for instance, many 3-star reviews are predicted as 2-star or 4-star, and 2-star reviews are often confused with 1-star or 3-star.
- The model has higher recall for classes 1 and 5, showing it captures strongly negative and strongly positive sentiments more effectively than neutral or mixed tones.

Overall Conclusion

Overall, all four models demonstrate a consistent trend, strong at identifying extreme sentiments (1-star and 5-star reviews) but limited in distinguishing neutral or mid-range sentiments.

Among them, Linear SVM and Logistic Regression achieved the best and most stable performance, making them the most suitable baselines for this sentiment classification task. Further improvements can be expected by enhancing feature representations and experimenting with advanced models like Gradient Boosting or Transformer-based classifiers.